

FOUNDER OPERATING SYSTEM

Phase 3 - Product Learning, QA, Release, and Customer Proof

Complete Technical Specification and Implementation Plan

FOS-TECH-PHASE-3 | Version 3.0 | 2026-07-13

Contents

- 0. Revision decision
- 1. Implementation directive
- 2. Objectives and metrics
- 3. Scope
- 4. Domain model
- 5. Workspace projections
- 6. Agents
- 7. Required test suites
- 8. Core workflows
- 9. APIs and jobs
- 10. Deterministic gates
- 11. Tests
- 12. Work packages
- 13. Definition of done
- Shared architectural invariants
- Agent runtime requirements
- Security, privacy, and governance
- Observability and cost controls
- Deployment and activation model

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 3 retains product learning and release governance while adding the customer-proof and release-communications layer that marketing needs. Product signals, requirements, tests, defects, and release gates remain canonical. Editable specification prose, release narratives, case studies, and technical papers use generic artifacts and Notion working copies.

1. Implementation directive

Build a traceable signal-to-release system that converts beta evidence into prioritized problems, implementation specifications, tests, release decisions, and approved customer proof.

2. Objectives and metrics

Reduce time from repeated beta problem to approved change decision.

Maintain requirement-to-test traceability.

Detect regressions, cross-user memory leakage, prompt injection, and approval failures.

Produce evidence-backed release-readiness reports.

Revalidate product and marketing claims after releases.

Convert verified releases and outcomes into approved release notes, case studies, and technical papers.

Metrics include signal-to-decision time, requirement coverage, regression detection, defect escape, release cycle time, blocked-release precision, QA cost, verified proof assets, and founder review time.

3. Scope

Included:

Product signals and clustering

Product change proposals

Versioned specification artifacts

Canonical requirements and acceptance criteria

Synthetic personas and tests

Test execution and evidence

Defects and regression investigation

Release candidates and readiness reports

Claim revalidation

Release notes, case studies, beta-learning reports, and architecture papers

Excluded:

Autonomous production deployment

Agent waiver of critical gates

Unsupported customer-causality claims

Full scaled campaign orchestration

4. Domain model

ProductSignal

Stores product area, signal type, statement, classification, source, affected count, severity, business impact, confidence, and review status.

SignalCluster

Stores approved problem statement, signal links, affected segments, frequency, enrollment/retention/founder-time impact, confidence, and disposition.

ProductChangeProposal

Stores proposed change, scope/non-scope, expected value, risk, effort, priority, decision, and specification reference.

SpecificationRecord

Stores canonical metadata and references an artifact for editable prose. Canonical fields include spec key, version, status, evidence manifest, approved goals/non-goals, requirement IDs, approval, and supersession.

RequirementRecord

Stores key, type, description, priority, risk, acceptance criteria, implementation references, status, and tests.

SyntheticPersona

Stores goals, constraints, starting data, behavior profile, adversarial traits, and expected boundaries.

TestCase and TestRun

Store requirement links, type, preconditions, steps, expected/evaluation rules, severity, automation, environment, actual result, failure classification, evidence, cost, and latency.

Defect

Stores reproduction, expected/actual behavior, severity, source signals/runs, failure classification, fix reference, and verification.

ReleaseCandidate

Stores version, specifications, requirements, implementation refs, model/prompt/memory/migration changes, known limitations, rollback, and status.

ReleaseReadinessReport

References an artifact for the narrative and stores canonical requirements/test/defect/security summaries, recommendation, confidence, blockers, and approval.

ProofAsset

Links a release, outcome evidence, claim set, consent, and a content artifact such as case study, release note, technical paper, or public beta-learning report.

5. Workspace projections

Product Signals

Founder reviews clusters, evidence, impact, and disposition.

Product Specifications

Artifact body is editable in Notion. Requirement IDs, test coverage, approval, and version remain canonical.

QA and Release Center

Projects suite summaries, blockers, known limitations, claim impact, release narrative, and founder decision. Detailed test telemetry remains in FOS.

Customer Proof Queue

Projects outcome/release evidence eligible for a case study, release note, or technical paper, including consent and claims status.

6. Agents

fos.product_signal_synthesizer

fos.change_proposal

fos.beta_change_spec_compiler

fos.specification_critic

fos.test_planner

fos.synthetic_user

fos.regression_investigator

fos.release_readiness

fos.claim_impact_analyzer

fos.release_communications

fos.case_study_builder

fos.technical_paper_builder

The release and proof agents may draft narratives but may not change release gates, consent, or claim approval.

7. Required test suites

Critical journeys

Application, enrollment, onboarding, first value, support, roadmap, resume/LinkedIn, portfolio, interview, shared memory, content approval, and workspace commands.

Cross-module consistency

Resume claims align with portfolio evidence.

Interview coaching reflects demonstrated skills.

Roadmap reflects assessment and updated goal.

Enrollment promises match current capability.

Marketing claims match deployed release.

Public proof matches consent and evidence.

Security and memory

Cross-user isolation

Inference versus confirmed memory

Supersession and audit

Prompt injection through applications, documents, support, workspace pages, and web content

Unauthorized tool and approval attempts

8. Core workflows

Signal to approved change

Collect signals, cluster, founder approves problem, create proposal, generate spec artifact, run critic, founder approves, create requirements/tests, and project implementation package.

Test and regression

Create/run suite, store evidence, classify failures, create defects, verify fixes, compare model/prompt versions, and preserve baseline.

Release decision

Create candidate, run required suites, generate readiness report, identify claim/content impact, founder approves or blocks, record deployment separately, and update projections.

Proof generation

After deployment and verified outcome/release evidence, create proof candidate, verify claim and consent, generate case study/release note/paper, founder approves, and hand approved artifact to Phase 4 campaign operations.

9. APIs and jobs

API families:

/api/fos/product-signals/*

/api/fos/signal-clusters/*

/api/fos/change-proposals/*

/api/fos/specifications/*

/api/fos/requirements/*

/api/fos/test-cases/*

/api/fos/test-runs/*

/api/fos/defects/*

/api/fos/release-candidates/*

/api/fos/proof-assets/*

Jobs:

cluster-product-signals

generate-change-proposal

generate-specification-artifact
generate-test-plan
run-regression-suite
run-memory-isolation-suite
run-prompt-injection-suite
investigate-regression
generate-release-readiness-report
analyze-release-claim-impact
generate-release-communications
generate-proof-asset

10. Deterministic gates

Approved requirements must link to tests before release-ready status.
Critical security, privacy, memory, or approval failures block release.
Agents cannot mark tests accepted or waive blockers.
Deployment is a separate founder-controlled action.
Release changes invalidate or revalidate affected claims.
Customer proof requires source verification and consent.
Notion may edit narrative artifacts, not test results or release gates.

11. Tests

Unit tests cover cluster scoring, requirement/test links, release-gate rules, claim impact, proof eligibility, artifact versioning, and workspace conflicts.
Integration tests cover support signal to defect, approved problem to specification, test failure to defect, release block, claim invalidation, and consent block.
End-to-end tests cover signal to release, prompt injection, cross-user memory isolation, founder-edited specification, release note generation, and case-study approval.

WP3.0

Phase 2 signal and outcome handoff

WP3.1

Signal and cluster registry

WP3.2

Change proposals and decisions

WP3.3

Specification artifacts and canonical requirements

WP3.4

Synthetic personas and test registry

WP3.5

Test execution and evidence

WP3.6

Security, memory, and injection suites

WP3.7

Defects and regression investigation

WP3.8

Release candidates and readiness

WP3.9

Claim impact and revalidation

WP3.10

Release communications and customer proof

WP3.11

Product/QA/Release Notion workspaces

13. Definition of done

Product decisions are traceable to beta evidence.

Specification prose is versioned while requirements/tests remain canonical.

Critical journeys and security boundaries are tested.

Agents cannot waive release gates or deploy.

Release changes revalidate claims.

Approved proof assets are evidence- and consent-backed.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are canonical_read_only, working_copy_editable, controlled_command, or not_projectable.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 3 - Product Learning, QA, Release, and Customer Proof according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Requirements, tests, defects, release gates, and deployment state remain canonical.

Specification and release narratives use generic artifact versions and controlled founder edits.

Critical security, privacy, memory-isolation, or approval failures block release.

Agents may recommend release status but may not waive gates or deploy.

Customer proof requires verified evidence, consent, and claims validation. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.